

Nathan Joseph Svoboda

630-532-9604 | njsvoboda04@gmail.com | Tempe, Arizona | ennjaywithagreyhat.github.io | github.com/EnnJayWithAGreyHat

Education

Bachelor of Science, Computer Science at *Barrett the Honors College at Arizona State University* August 2022 - May 2026
- Awards: Dean's List 7 Semesters (3.5 GPA or above)

Technical Skills

Programming Languages: C, C++, Java, Python, JavaScript, HTML, CSS, SQL, Bash
Frameworks: React, Gradle, Baja, Niagara, Unix, Pandas, Numpy, TensorFlow, Djitellopy, OpenCV, Matplotlib, Keras
Project Management: Git, Scrum, ClickUp

Professional Experience

Full-Stack Software Engineering Intern - Hawkeye Energy Solutions May 2024 - August 2025

- Designed new React components for a customer-facing front-end application by leveraging state management, HTML, and CSS, allowing customers to set maximum and minimum settings through a user-friendly design
- Redesigned the backend of a React application to fix API call and data resolution errors by fixing Java methods that would communicate directly with building meters through integrated servers
- Spearheaded the migration and integration of historical metering data from an older database to a newer and more dynamic database using SQL to provide building metering trends from the past for a client
- Currently working in a CI/CD environment for multiple customer-facing applications utilizing npm, Gradle, GitHub, Java, and the Niagara framework to maintain and develop applications and documentation for customers

Software Team Lead - Paragon Autonomous January 2024 - May 2025

- Built software for an autonomous fire surveillance drone using Python and TensorFlow, allowing it to fly with manual controls, notify the host computer when prompted, and visualize its position using the Pygame library
- Implemented A* algorithm to find the best path using point clouds in 3D space and with computation restrictions using C++ to provide autonomous movement and navigate to the site of a potential fire hazard when found
- Researched and developed a template framework in C++ to simplify code deployment with flight controllers, and ensured compatibility with Mavros, Gazebo, and Ardupilot for drone programming and functionality
- Tested the Lidar and Ultrasonic sensors to see the information that is provided, and how that data can be integrated into our code for adaptation of flight controls, and collision avoidance

Projects

Personal Website | React, HTML, CSS, Javascript, SQL August 2024 - Present

- Built a responsive personal portfolio using React, JavaScript, HTML, and CSS and a D3 visualization to demonstrate Dijkstra's and Prim's algorithms, allowing visitors to explore shortest-path behavior
- Integrated EmailJS contact form and optional Google OAuth hook, managed builds with npm and published via GitHub Pages to streamline deployment and ensure accessibility for various browsers and browser sizes
- Provisioned and configured a MongoDB database and implemented connection logic for backend storage, live integration failed during deployment due to hosting/connection constraints, so I documented troubleshooting

TensorFlow Models and Learning | Keras, Pandas, OpenCV, TensorFlow Jupyter Notebooks August 2024 - Present

- Currently learning to train and code machine learning models using the Pandas, Keras, and TensorFlow libraries
- Building a model using OpenCV that can categorize images from a camera that can be used to determine navigational and fire-prone hazards for Paragon Autonomous drones
- Organized all of the important code files using Jupyter Notebooks and Google Colab to develop working code

Dijkstra's and Prim's Algorithm | C++, GitHub April 2024

- Wrote Dijkstra's Algorithm from scratch in C++ and learned how to implement a BFS, Queue, Min Heap, write efficient structs, and assemble binaries for the functionality of the program that would work on a Matrix Graph
- Learned to implement Prim's Algorithm, Makefiles, memory management with Valgrind, and debugging with GDB
- Built a graph into my portfolio website using the d3 library to show how my code evaluates the shortest path to simplify and demonstrate the complexity